

Exercício 01

Para cada alternativa, fornecemos o MGU (most general unifier) e a lista de termos resultante:

a) resolution([p,q,r],[p,x,y])

MGU: {x=q, y=r}

Goals Resultantes: [q, q, r]

b) resolution([p(a)], [p(X)])

MGU: {X=a}

Goals Resultantes: [p(a)]

c) resolution([p(X,b),q(X)], [p(a,Y), r(Y)])

MGU: {X=a, Y=b}

Goals Resultantes: [(q(a), r(b))]

d) resolution([append([],B,B)], [append(X,Y,[1,2])])

MGU: {X=[], Y=[1,2], B=[1,2]}

Goals Resultantes: [append([], [1,2], [1,2])]

Exercício 02

Consulta: append(X, Y, [1,2])

Prova Completa:

Consulta inicial: append(X, Y, [1,2])

Combina com a segunda cláusula: X = [Head|TailA] e TailC = [1,2]

Substituições parciais: Head = 1, TailC = [2]

Nova consulta: append(TailA, Y, [2])

Combina novamente com a segunda cláusula: X = [Head|TailA] e TailC = [2]

Substituições parciais: Head = 2, TailC = []

Nova consulta: append(TailA, Y, [])

Combina com a primeira cláusula: append([], Y, Y)

Resultado: X = [], Y = [1,2]

Soluções:

Solução 1: X = [], Y = [1,2]

Solução 2: X = [1], Y = [2]

Solução 3: X = [1,2], Y = []

Exercício 03

Um programa em Prolog com uma árvore de provas finita pode executar para sempre?

Sim. Isso ocorre quando há recursão infinita sem uma condição de parada explícita.

Exemplo: loop :- loop.

Consulta: loop.

A recursão nunca termina, mesmo que a árvore de provas seja simples.

Um programa em Prolog que não executa para sempre pode ter uma árvore de provas infinita?

Sim. Isso ocorre se a árvore de provas contém caminhos infinitos, mas a consulta termina antes de explorar todos os ramos.

Exemplo:

fact(0).

fact(N) :- N > 0, M is N - 1, fact(M).

Consulta: fact(0).

O programa termina rapidamente, mas a árvore de provas para valores maiores de N é infinita devido à recursão.

Exercício 04

Novas Cláusulas para o Jogo

```
/*
```

```
  Adiciona o portão como local intermediário entre a bifurcação e o topo da montanha.
```

```
*/
```

```
connect(fork, forward, gate).
```

```
connect(gate, forward, mountaintop).
```

```
/*
```

```
  Inicializa o estado do portão (trancado por padrão) e a posição da chave.
```

```
*/
```

```
:- dynamic locked/1.
```

```
locked(gate).
```

```
at(key, maze(1)).
```

```
/*
```

```
  Mecanismo para pegar a chave.
```

```
*/
```

```
take(key) :-
```

```
  at(you, Loc),
```

```
  at(key, Loc),
```

```
  retract(at(key, Loc)),
```

```
  assert(at(you, key)),
```

```
  write('You picked up the key.\n'),
```

```
  !.
```

```
take(key) :-
```

```
  write('There is no key here.\n').
```

```
/*
```

```
  Mecanismo para destrancar o portão.
```

```
*/
```

```
unlock(gate) :-
```

```
  at(you, gate),
```

```
  at(you, key),
```

```
  retract(locked(gate)),
```

```
  write('You unlocked the gate.\n'),
```

```
  !.
```

```
unlock(gate) :-
```

```
  write('You cannot unlock the gate.\n').
```

```
/*
```

```
  Controle para passar pelo portão segurando a chave.
```

```
*/
```

```
move(forward) :-
```

```
  at(you, gate),
```

```
at(you, key),
write('A lightning bolt strikes you as you try to pass through the gate holding the key.\n'),
retract(at(you, gate)),
assert(at(you, done)),
!.
```

```
/*
```

```
    Passar pelo portão sem a chave.
```

```
*/
```

```
move(forward) :-
```

```
    at(you, gate),
    \+ locked(gate),
    retract(at(you, gate)),
    assert(at(you, mountaintop)),
    report,
    !.
```